

OrderedDict

When we perform insertions into a standard Dictionary the order is not retained and we end up getting elements in an arbitrary order when we iterate over it. There might be use cases where we wish to preserve the order of insertion and retrieve elements in the same order as they were inserted. An `OrderedDict` is a perfect choice for such scenarios, generally called an ordered dictionary. It is a variation and a subclass of the `dict`, supporting the usual `dict` methods and a few more. Its constructor has a similar signature as a `dict` and returns an instance of a `dict` subclass.

Now that we know what exactly an ordered dictionary is, we will look at a few of its quirks.

- If we assign a new value to a key, the position of the key remains unchanged

```
>>> od = OrderedDict({'a': 1, 'b': 2, 'c': 3})
>>> for key, value in od.items():
...     print(key, value)
...
a 1
b 2
c 3
>>> od['a'] = 7
>>> for key, value in od.items():
...     print(key, value)
...
a 7
b 2
c 3
```

- If we delete a key and reinsert the same key into `OrderedDict`, the key will be pushed to the end, since it is a newly inserted key. Let's use the same example as above.

```
>>> od = OrderedDict({'a': 1, 'b': 2, 'c': 3})
>>> del od['a'] #delete key 'a'
>>> for key, value in od.items():
...     print(key, value)
...
b 2
c 3
>>> od['a'] = 1 #add key 'a'
>>> for key, value in od.items():
```